Alternatively, you can register the `IOptions` type in the DI container but then use a lambda to additionally register `MapSettings` as a singleton so it can be directly injected, as shown in listing 10.12.

Listing 10.12 Configuring strongly typed settings for direct injection

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.Configure<MapSettings>(                          ❶
    builder.Configuration.GetSection("MapSettings"));             ❶
builder.Services.AddSingleton(provider =>                         ❷
    provider.GetRequiredService<IOptions<MapSettings>>().Value);  ❷

WebApplication app = builder.Build();

app.MapGet("/", (MapSettings mapSettings) => mapSettings);        ❸

app.Run();
```

❶ Configures the IOptions as normal

❷ Registers the MapSettings object in DI by delegating to the IOptions registration

❸ Injects the MapSettings object directly DI

If you use either of these approaches, you won't benefit from the ability to reload strongly typed settings without further work or from some of the more advanced uses of `IOptions`, but in most cases, that's not a big problem. I'm a fan of these approaches generally, but as always, consider what you're losing before adopting them wholeheartedly.

TIP In chapter 31 I show one such advanced scenario in which you configure an `IOptions` object using services in your DI container. For other scenarios, see Microsoft's "Options pattern in ASP.NET Core" documentation at http://mng.bz/DR7y, or see the various `IOptions` posts on my blog, such as this one: http://mng.bz/l1Aj.

That brings us to the end of this section on strongly typed settings. In section 10.4 we'll look at how you can change your settings dynamically at runtime, based on the environment in which your app is running.